

SKILL WEEK7

TASK 1: Process Synchronization and Child Process Monitoring Using waitpid()

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid, result;
    int status;

    printf("Parent process started. PID = %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("Child process started. PID = %d\n", getpid());
        printf("Child is working...\n");

        sleep(3);

        printf("Child process completed.\n");
        exit(10);
    }
    else {
        // Parent process
        printf("Parent is waiting for child process...\n");

        result = waitpid(pid, &status, 0);

        if (result == -1) {
            perror("waitpid failed");
            return 1;
        }
    }
}
```

```

    }
    else {
        // Parent process
        printf("Parent is waiting for child process...\n");

        result = waitpid(pid, &status, 0);

        if (result == -1) {
            perror("waitpid failed");
            return 1;
        }

        if (WIFEXITED(status)) {
            printf("Child process terminated normally.\n");
            printf("Child exit status = %d\n", WEXITSTATUS(status));
        }

        printf("Parent process completed.\n");
    }

    return 0;
}

```

```

laasya@laasya: ~
laasya@laasya:~$ nano task1.c
laasya@laasya:~$ gcc task1.c -o task1
laasya@laasya:~$ ./task1
Parent process started. PID = 2453
Parent is waiting for child process...
Child process started. PID = 2454
Child is working...
Child process completed.
Child process terminated normally.
Child exit status = 10
Parent process completed.

```

TASK 2: To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

laasya@laasya: ~



GNU nano 7.2

task2.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

#define MAX_PATHS 100

int main(int argc, char *argv[]) {
    char *path;
    char *path_copy;
    char *directory;
    char full_path[1024];
    struct stat file_info;

    // Check whether a command was supplied
    if (argc != 2) {
        printf("Usage: %s <command>\n", argv[0]);
        return 1;
    }

    // Retrieve PATH variable
    path = getenv("PATH");

    if (path == NULL) {
        printf("PATH variable is not set.\n");
        return 1;
    }

    printf("PATH variable:\n%s\n\n", path);

    // Make a copy because strtok modifies the string
    path_copy = strdup(path);

    if (path_copy == NULL) {
        perror("strdup failed");
        return 1;
    }
}
```

```

printf("Searching for command: %s\n\n", argv[1]);

// Parse PATH directories
directory = strtok(path_copy, ":");

while (directory != NULL) {

    // Construct complete path
    snprintf(full_path, sizeof(full_path),
             "%s/%s", directory, argv[1]);

    printf("Checking: %s\n", full_path);

    // Check whether file exists
    if (stat(full_path, &file_info) == 0) {

        // Check whether it is a regular file
        if (S_ISREG(file_info.st_mode)) {

            // Check execute permission
            if (access(full_path, X_OK) == 0) {
                printf("\nCommand found!\n");
                printf("Executable: %s\n", full_path);

                free(path_copy);
                return 0;
            }
        }

        directory = strtok(NULL, ":");
    }

    printf("\nCommand '%s' not found in PATH.\n", argv[1]);
}

```

```

laasya@laasya:~$ nano task2.c
laasya@laasya:~$ gcc task2.c -o task2
laasya@laasya:~$ ./task2 ls
PATH variable:
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr
/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/Common Files/Oracle/Java/
javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/
c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/windows/syste
m32:/mnt/c/windows:/mnt/c/windows/System32/Wbem:/mnt/c/windows/System32/Win
dowsPowerShell/v1.0/:/mnt/c/windows/System32/OpenSSH:/mnt/c/Program Files/do
tnet:/mnt/c/Program Files/nodejs:/mnt/c/Users/laasy/AppData/Local/Microsof
t/WindowsApps:/mnt/c/Users/laasy/AppData/Local/Programs/Microsoft VS Code/bi
n:/mnt/c/Users/laasy/AppData/Local/Python/bin:/mnt/c/Users/laasy/AppData/Roa
ming/npm:/snap/bin

Searching for command: ls

Checking: /usr/local/sbin/ls
Checking: /usr/local/bin/ls
Checking: /usr/sbin/ls
Checking: /usr/bin/ls

Command found!
Executable: /usr/bin/ls

```